Hyperparameter fine tune

Lr0 test range from 0.0001, 0.0005, 0.001 and 0.005

Default hyperparameters besides Lr0 as follows, all except Lr0 were kept the same.

model.train(data=config_path, epochs=5, patience=5, batch=16, workers=2, optimizer='AdamW', project=project_dir, name=f'lr0_{lr}', exist_ok=True)

Result

Lower learning rates did better at 5 epochs. Due to low dataset count, explicit AdamW runs, lr0 = 0.001 is the best, and the score drops steadily as lr0 increases. Yolo8 has default lr0_0.0, but I am using the lab 0.005 as a baseline.

Run	Optimizer	lr0	P	R	mAP50	mAP50-95	val box/ cls loss	F1
lr0_0.0001	AdamW	0.0001	0.731	0.799	0.823	0.492	1.117/0.924	0.763
lr0_0.0005	AdamW	0.0005	0.740	0.744	0.813	0.478	1.135/0.964	0.742
lr0_0.001	AdamW	0.001	0.726	0.750	0.790	0.464	1.160/0.960	0.738
lr0_0.005	AdamW	0.005	0.671	0.702	0.721	0.413	1.222/1.079	0.686

All numbers are from epoch 5, the final epoch, with seed=0 (Ultralytics default) for every run.

Results indicated lr0_0.0001 had the best performances mAP50 - 95 = 0.492. lr0_0.002 (0.478). As lr0 approaches 0.005, the score starts dropping. Lr0 = 0.01 had the worst score (0.384).

Surprisingly, lr0_0.001 has a slightly higher Recall rate than the baseline.

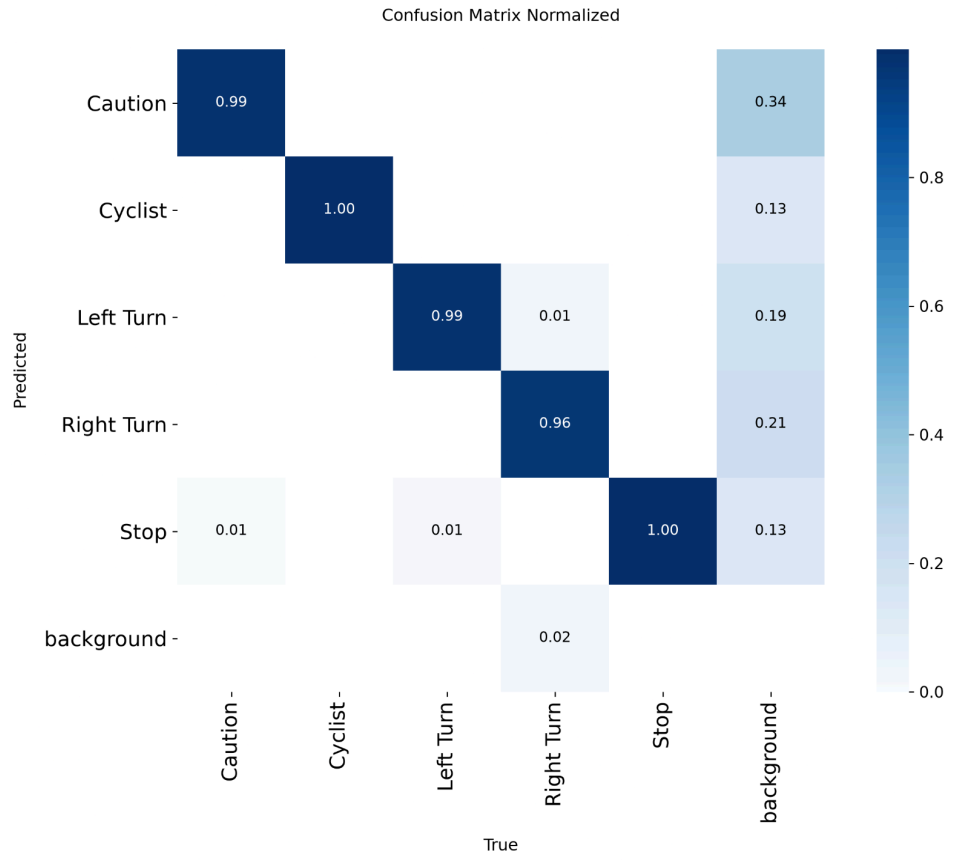
The lowest score overall is lr_0.005. Worth mentioning, the F1 score differences between lr0_0.0005 and lr0_0.001 was only 0.004. From the val box /cls loss score, this might be due to lower box drawing accuracy.

mAP50-95 per epoch

epoch	0.0001	0.0005	0.001	0.005
1	0.351	0.330	0.312	0.176
2	0.428	0.386	0.337	0.255
3	0.460	0.417	0.380	0.247
4	0.490	0.454	0.434	0.371
5	0.492	0.478	0.464	0.413

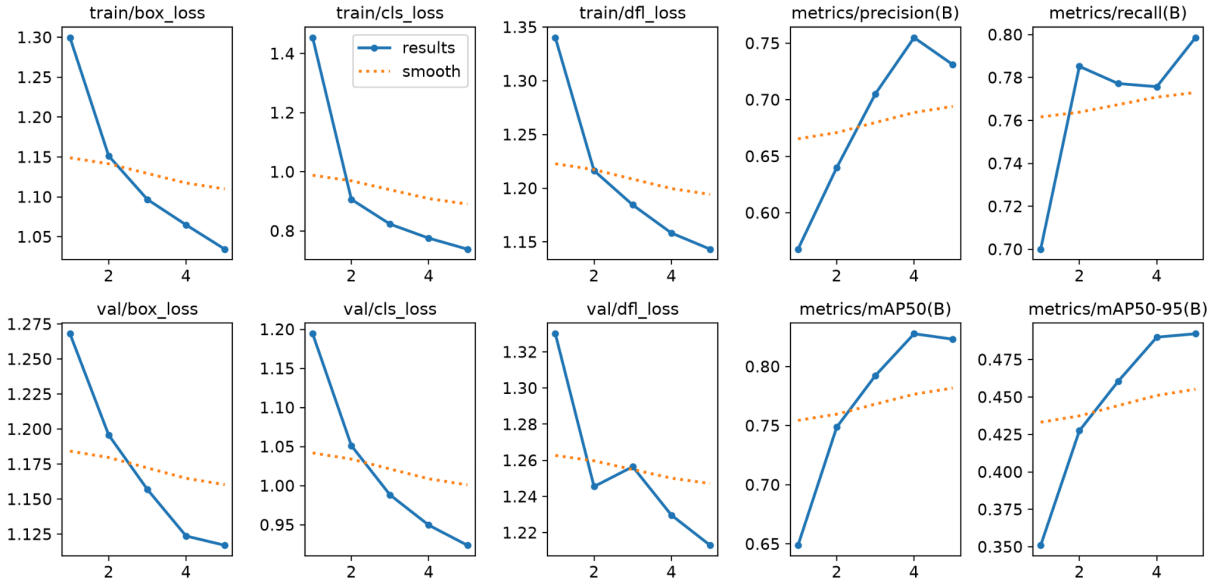
Lr0_0.001 was leading since the first epoch; score rising indicates that training might benefit from more epochs. The lr0_0.005 score dropped from 0.255 to 0.247 from 3th epoch to 4th, indicating the lr0 is too large.

Confusion Matrix Normalized



Normalized Confusion Matrix on the validation set showed 5 different categories have high prediction success with the exception of background (no cyclist) with high misses.

Training Curves from results.png lr0_0.0001



Lr0_0.0001 mAP50-95 = 0.492

From the graph, we can see box_loss, cls_loss, from both train and val are decreasing. mAP50 and mAP50-95 are showing signs of convergence at the 5th epoch.

Scores per class performance lr0_0.0001

Class	images	instances	Precision	Recall	mAP50	mAP50-95
All	1535	2256	0.730	0.799	0.823	0.493
Caution	426	447	0.521	0.564	0.620	0.345
Cyclist	756	1110	0.950	0.979	0.986	0.765
Left Turn	154	198	0.630	0.924	0.829	0.443
Right Turn	206	256	0.786	0.746	0.812	0.389
Stop	219	245	0.761	0.780	0.869	0.523

Cyclist was easy to identify, and Caution being the weakest given it has more instances than some other classes that achieved higher scores.

Limitations

Although lr0_0.0001 is showing as the best, it is only the smallest lr0 among other results. The most optimal one could be even smaller. Second, each value only ran for 5 epoch, with longer epoch runs, different lr0 settings might close in the gaps.

Suggestions

Longer epochs like 50 to inspect learning curves and other graphs for convergence.
Lr0_0.0001 might be the best, but lower lr0 could potentially raise the scores higher. Based on the confusion matrix, It's also worth investigating the dataset to find out what leads to poor background identification scores.